House Price Prediction

Introduction

This project is called House Price Prediction. The goal is to build a program that can guess the price of a house based on its details, such as how many bedrooms it has, the size of the house in square feet, which year it was built, and other similar information.

To do this, we used a method called Machine Learning, which means we gave the computer a large amount of house data and let it learn patterns on its own. We trained the model using seven different algorithms, which are seven different ways of teaching the computer to make predictions.

Before training, we used a technique called Feature Scaling (StandardScaler) to make sure all the numbers are on the same scale, so that features with very large numbers do not overpower features with small numbers.

Dataset used: kc_house_data.csv

The dataset was split into two parts. 80% of the data was used for training (so the model could learn), and the remaining 20% was used for testing (to check how well it learned).

Machine Learning Algorithms Used

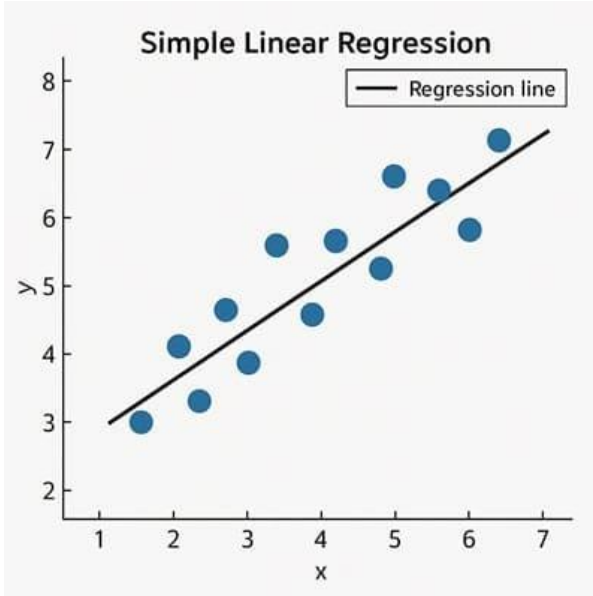
1. Linear Regression

This is the simplest algorithm. It tries to draw a straight line through all the data points that best represents the relationship between house features and price.

Think this, if a house has more square footage, the price goes up in a straight and predictable way. The algorithm finds the best straight line that fits this pattern.

Good for: Simple data where the relationship is straightforward.

Weakness: Does not work well when the relationship between features and price is complex or curved.



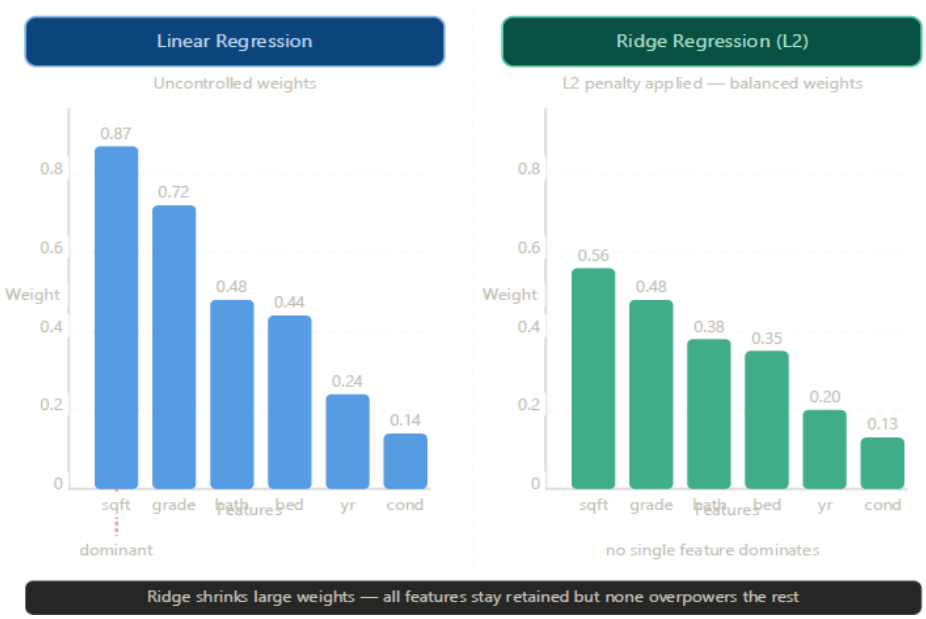
2. Ridge Regression

Ridge Regression is basically the same as Linear Regression, but with one extra safety rule. It adds a penalty to stop any one feature from having too much influence over the prediction.

For example, without this penalty, the algorithm might rely too heavily on square footage and almost ignore all other features. Ridge Regression keeps things more balanced.

Good for: Datasets where many features are closely related to each other.

Weakness: It keeps all features, even the ones that are not very useful.



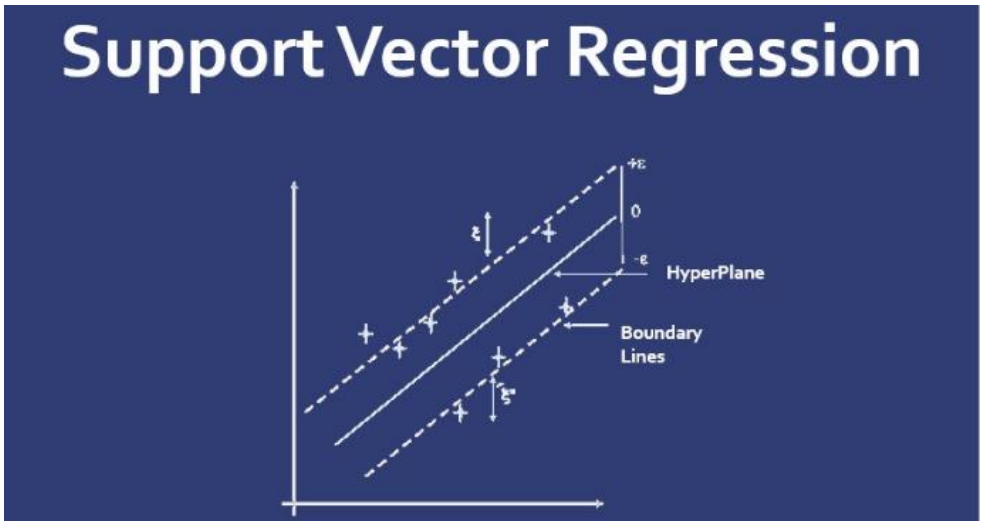
3. Support Vector Regressor (SVR)

Support Vector Regressor is a more advanced algorithm. Instead of finding a single best-fit line, it tries to fit a prediction line inside a tube or band. Any data point that falls inside the tube is considered a good enough prediction and is ignored.

Only the points that fall outside the tube are used to shape the line. This makes the model more focused and less affected by small random variations in data.

Good for: Complex data with unusual patterns.

Weakness: Needs careful tuning of settings and requires feature scaling to work properly.



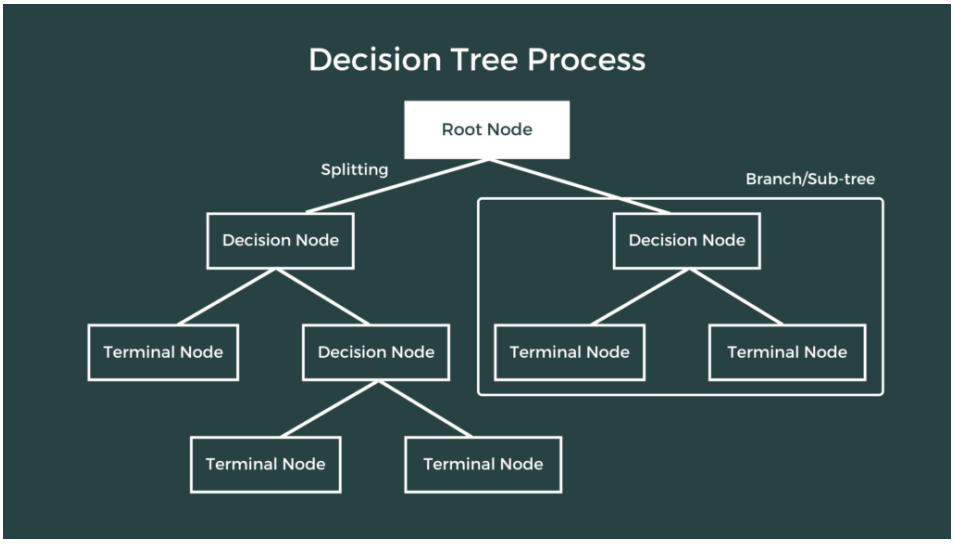
4. Decision Tree Regressor

A Decision Tree asks a series of yes or no questions about the house features to arrive at a predicted price. Each question narrows down the answer step by step, like a game of 20 questions.

For example: Is the house bigger than 2000 sqft? Yes. Does it have more than 3 bedrooms? No. Based on these answers, the predicted price is $240,000.

Good for: Non-linear data and easy to understand visually.

Weakness: If the tree is allowed to grow too deep, it memorizes the training data and performs poorly on new data. This is called overfitting.



5. Random Forest Regressor

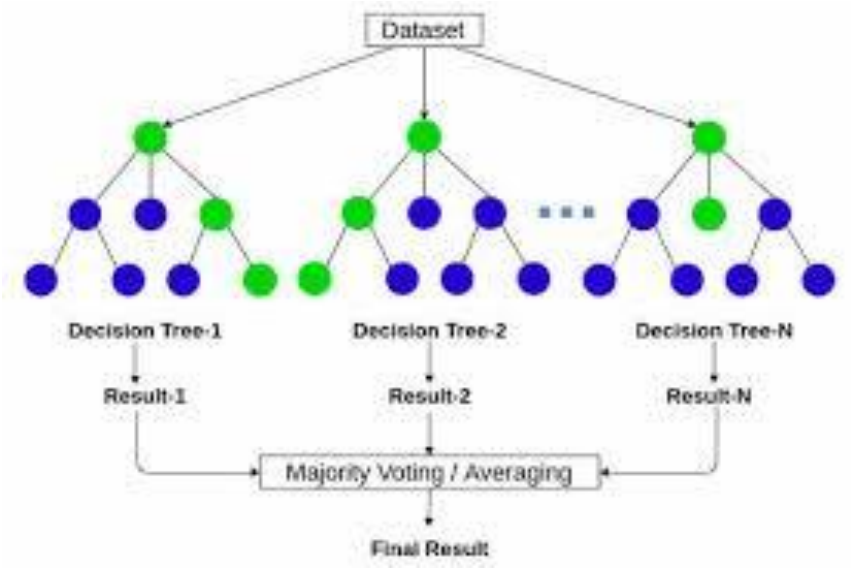
Random Forest builds many Decision Trees at the same time, each trained on a slightly different random portion of the data. At the end, it averages all the predictions from every tree to get the final answer.

This is more reliable than a single Decision Tree because the errors of individual trees cancel each other out when averaged together.

Easy version: Instead of asking one person for their opinion, you ask 100 people and take the average answer.

Good for: Most types of data. One of the most reliable general-purpose algorithms.

Weakness: Slower than a single Decision Tree and harder to interpret.



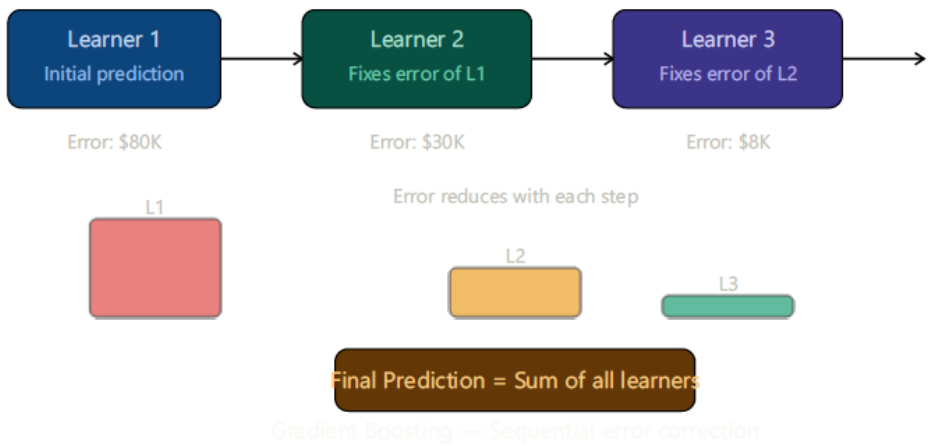
6. Gradient Boosting Regressor

Gradient Boosting also builds many Decision Trees, but in a different way than Random Forest. Instead of building all trees at the same time independently, it builds them one by one. Each new tree tries to fix the mistakes made by the previous tree.

Think of it like a student who keeps reviewing the questions they got wrong and improving on them with every practice session.

Good for: Getting very high accuracy results.

Weakness: Can be affected badly by noisy or messy data if the settings are not adjusted carefully.



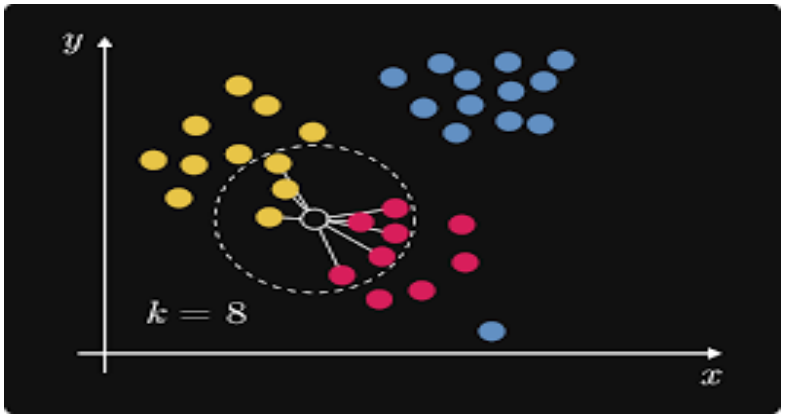
7. K-Nearest Neighbours Regressor (KNN)

KNN is the most intuitive algorithm of all. To predict the price of a new house, it simply looks at the K most similar houses in the training data and averages their prices.

KNN does not actually learn anything during training. It just stores all the data and searches through it when needed. This is why it is called a lazy learner.

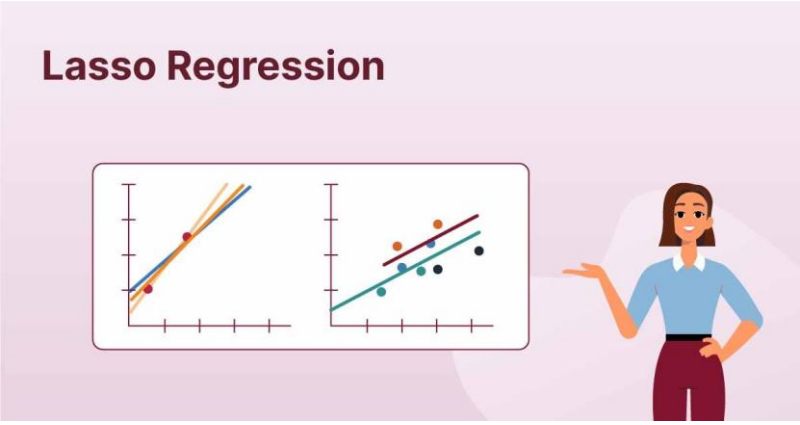
Good for: Small datasets where feature scaling has been applied.

Weakness: Gets very slow with large datasets because it must search through everything every single time.



8. Lasso Regression:

Lasso Regression is just like a linear regression that adds L1 regularization to improve Model generalization by adding a penalty. This penalty not only controls overfitting but Also performs feature selection by shrinking some coefficients to completely zero, Which means it automatically removes the features it considers less important from the model. It is particularly useful when the dataset has many features and only a few of them are contributing to the prediction

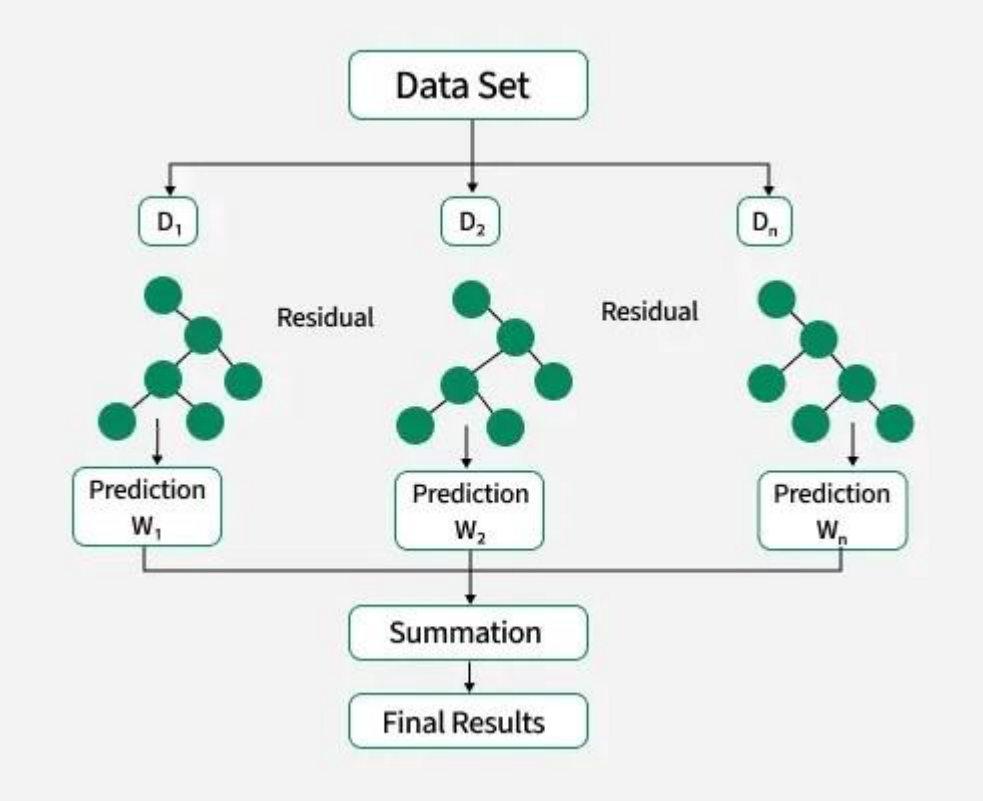


9. Bagging Regressor

Bagging Regressor is an ensemble learning method that stands for Bootstrap Aggregating. It combines multiple base regression models (typically decision trees) to improve model performance and reduce variance. It works by creating multiple random subsets of the training data through a process called bootstrap sampling, where data is sampled with replacement. Model is trained on each of these subsets independently and in parallel. The final prediction is made by averaging the outputs of all individual models.

10.XGBoost Regressor:

XGBoost stands for Extreme Gradient Boosting, it is an upgraded version of Gradient Boosting that builds weak learners sequentially, where each new learner focuses on correcting the errors of the previous one. XGBoost is significantly faster than regular Gradient Boosting because it supports parallel processing. It also has built-in L1 and L2 regularization which automatically prevents overfitting and it can also handle missing values on its own without requiring any extra preprocessing. These combined improvements make XGBoost one of the most accurate and efficient algorithm.



Conclusion

This project successfully trained and compared seven different machine learning algorithms on real house price data. All seven algorithms were tested on the same dataset and evaluated using three standard performance measures: R2 Score, MAE (Mean Absolute Error), and RMSE (Root Mean Squared Error).

The results showed that ensemble methods like Random Forest and Gradient Boosting gave the best and most reliable accuracy. Linear models like Linear Regression and Ridge Regression worked reasonably well but could not capture complex patterns. Tree-based models were better at handling non-linear relationships in the data.

The key takeaway is that no single algorithm is best for every situation. The right choice depends on the data and the specific goal of the project.

What I Learned

This project taught me several important lessons as a beginner in machine learning:

- Every algorithm has strengths and weaknesses depending on the data.
- Trying multiple algorithms on the same dataset is the best way to understand how differently each one thinks and behaves.
- Choosing the wrong algorithm can lead to underfitting (model is too simple and misses patterns) or overfitting (model memorizes training data and fails on new data).
- Evaluation metrics like R2 Score, MAE, and RMSE are essential tools. They take the complex performance of a model and reduce it to a single easy-to-read number.
- Feature scaling is a critical pre-processing step, especially for algorithms like SVR and KNN that are sensitive to the size of numbers.

Overall, this project gave me a solid foundation in supervised machine learning and taught me to think carefully before choosing a model, rather than just picking the first one that comes to mind.